Universidad Nacional Autónoma de México
Facultad de Ingeniería
Arquitectura Cliente Servidor
Ortega de la Paz Rafael | 420054085
18 de Octubre del 2026

Examen Parcial: Par de programas Cliente-Servidor

Introducción

En el presente trabajo se documenta la modificación del par de programas 04 visto en clase, un servidor y un cliente que se comunican mediante sockets de Internet. En el programa original el servidor envía un mensaje al cliente y ambos terminan; en esta versión el cliente envía primero su nombre leído desde el teclado, el servidor lo recibe, lo imprime junto con la IP desde la que se conectó el cliente y le responde con un mensaje. Se muestra la evidencia de la prueba local y de la conexión al servidor del profesor.

Modificaciones al código

En el servidor, el puerto dejó de estar definido en el código con #define PORT 4898 y ahora se recibe desde la línea de comandos con atoi(argv[1]); si no se proporciona, el programa muestra su modo de uso. El valor se asigna con htons(), que convierte el número al orden de bytes de red, necesario porque la computadora utilizada (Apple Silicon) almacena los enteros en formato little-endian y la red utiliza big-endian (Stevens et al., 2004).

Para capturar la IP del cliente se aprovecha que la función accept() guarda su dirección en la estructura cliente; el campo cliente.sin_addr se convierte a texto con inet_ntoa(). El nombre del equipo obtenido con gethostbyaddr() solo se imprime cuando la función no devuelve NULL, ya que de lo contrario el programa terminaría con un error de segmentación. Después de aceptar la conexión, el servidor primero hace recv() para recibir el nombre, lo imprime y luego responde con send().

En el cliente, el nombre se lee desde el teclado como en el programa 05: se limpia el búfer con memset(), se lee con fgets() y se elimina el salto de línea que deja la tecla

Enter. Una vez establecida la conexión con connect(), el cliente envía el nombre con send(), incluyendo el carácter nulo final, y después recibe la respuesta del servidor con recv(). En ambos programas se agregó la terminación de las cadenas recibidas con el carácter nulo y la revisión de errores con perror() en cada llamada de red (Kerrisk, 2010).

Actividades

Primero se verificó el entorno de trabajo con los comandos whoami, hostname, pwd, ls -l, sw_vers y cc --version, que muestran el usuario rafa en el equipo rafas-Mac-mini, la carpeta de trabajo con los archivos examen_cliente.c y examen_servidor.c, el sistema macOS 27.0.1 y el compilador Apple clang 21.

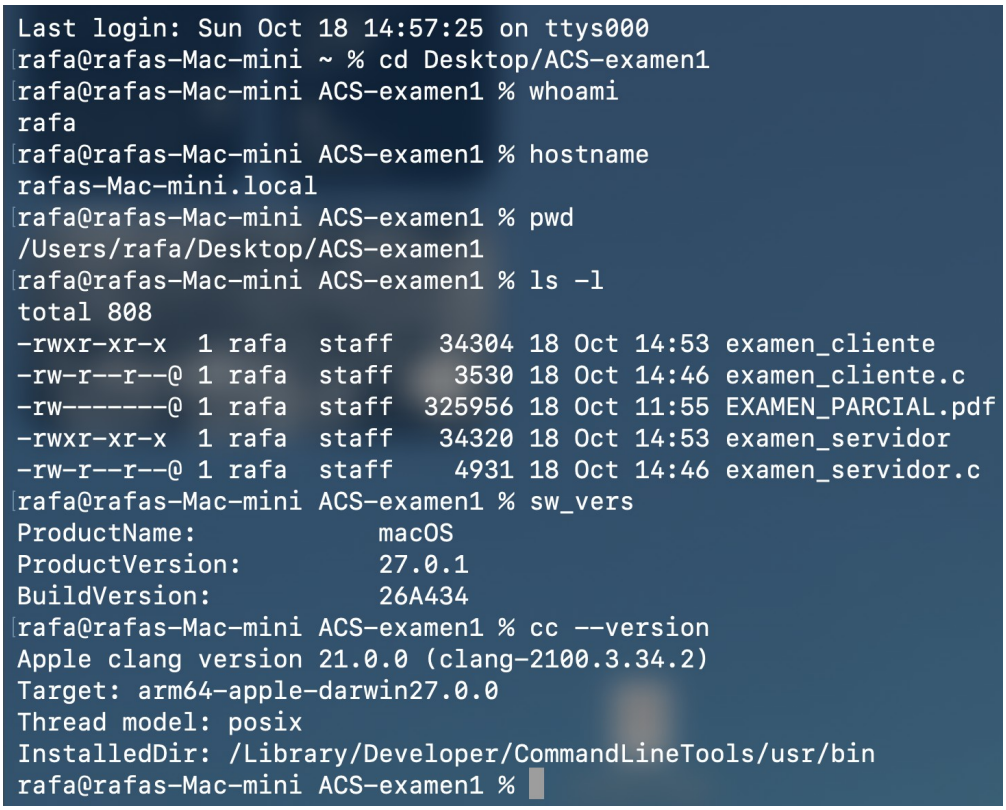


Imagen 1: Entorno de trabajo

Ambos programas se compilaron con cc -Wall -Wextra, opciones que activan las advertencias del compilador. Las dos compilaciones terminaron sin errores ni advertencias y se generaron los ejecutables examen_servidor y examen_cliente.

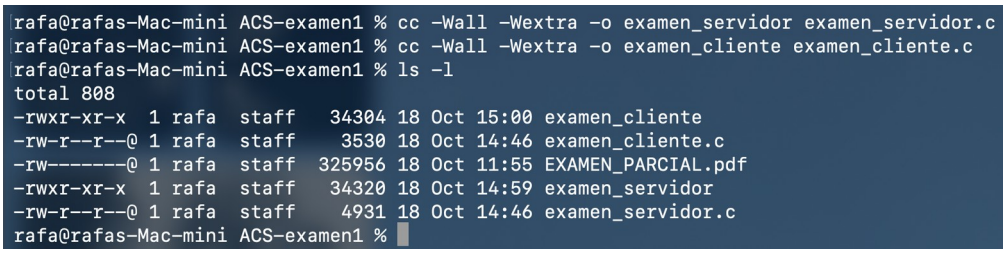


Imagen 2: Compilación de los programas

En el primer intento de prueba local se ejecutó el servidor en el puerto 5000, pero la función bind() falló con el mensaje Address already in use. Esto ocurre porque en macOS el puerto 5000 está ocupado por el servicio Receptor AirPlay del sistema. Gracias a la revisión de errores el problema se identificó de inmediato, y como el puerto se recibe desde la línea de comandos bastó con ejecutar el servidor en el puerto 4898, sin modificar el código.

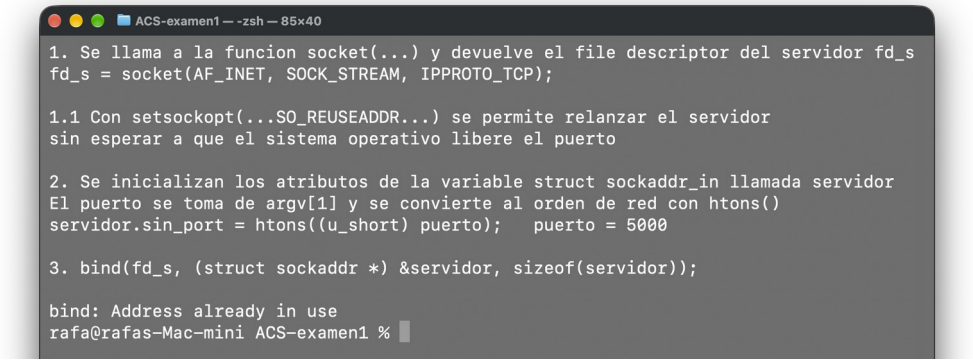


Imagen 3: Error al intentar usar el puerto 5000

Para la prueba local se ejecutó en una terminal el servidor con ./examen_servidor 4898 y en otra el cliente con ./examen_cliente 127.0.0.1 4898. En la terminal del servidor se observa que el puerto se recibió desde la línea de comandos, que la IP capturada del cliente es 127.0.0.1 y que el nombre recibido fue Ortega de la Paz Rafael. En la terminal del cliente se observa el nombre escrito desde el teclado y la respuesta del servidor.

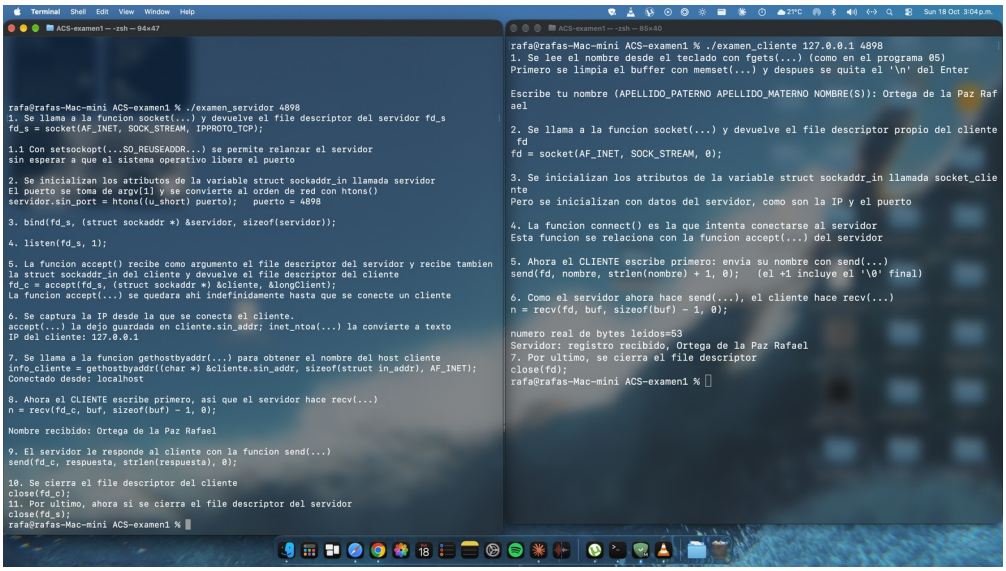


Imagen 4: Prueba local con ambas terminales

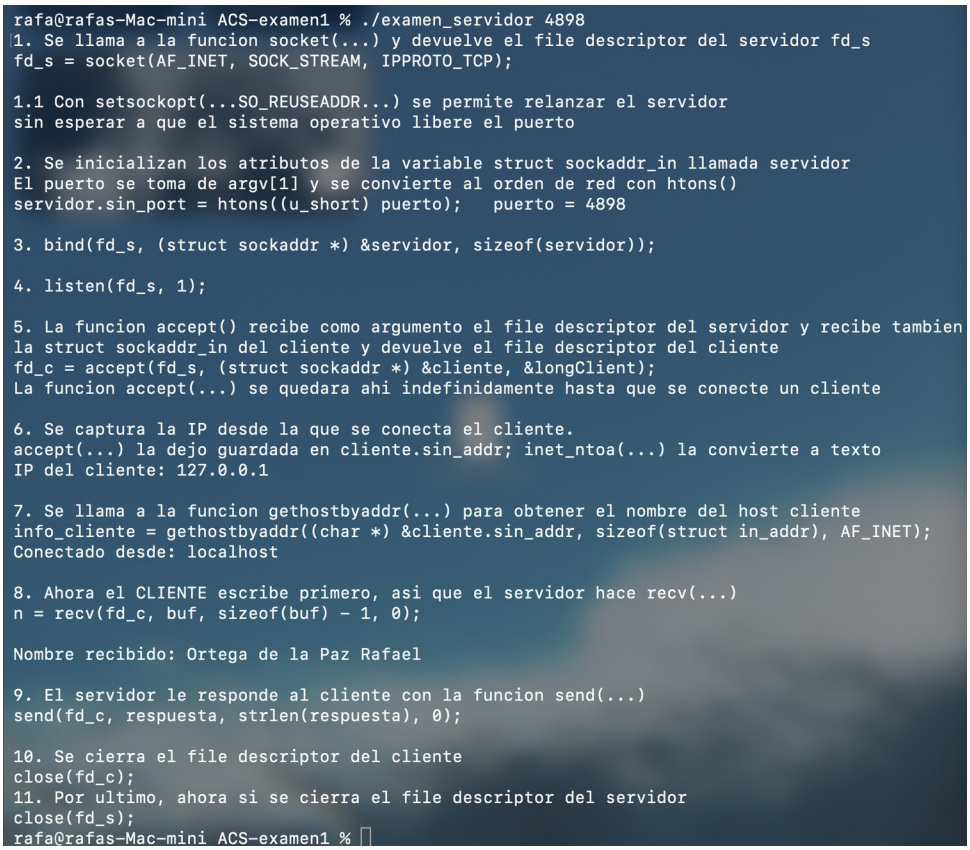


Imagen 5: Terminal del servidor

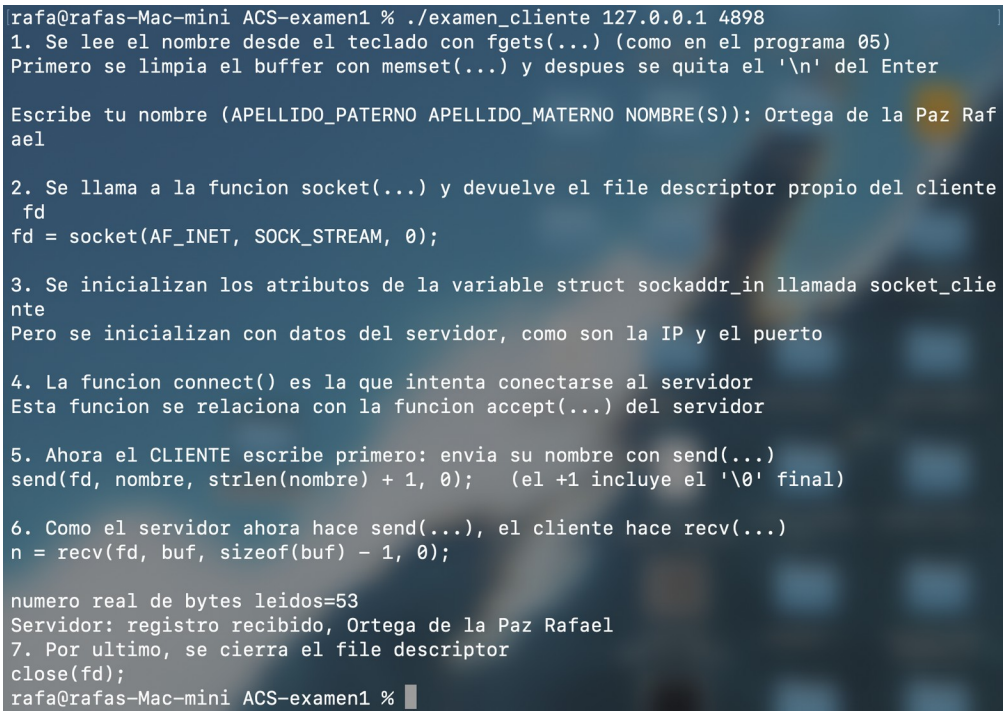


Imagen 6: Terminal del cliente

Por último, después de verificar la fecha y hora del equipo, se ejecutó el cliente contra el servidor del profesor con ./examen_cliente 132.248.59.6 4898 y se envió el nombre Ortega de la Paz Rafael. El servidor respondió con el mensaje Registraste exitosamente tu nombre!, y al terminar el cliente se ejecutó el comando date en la misma terminal, que registró la fecha y hora del envío: domingo 18 de octubre de 2026 a las 15:08:33.

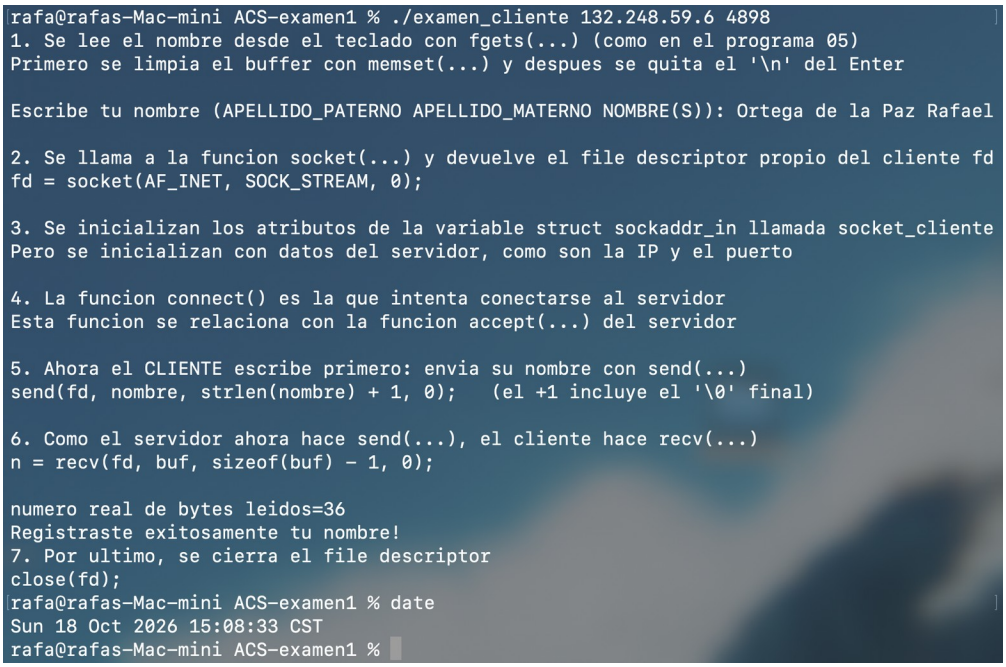


Imagen 7: Conexión al servidor 132.248.59.6 en el puerto 4898 y comando date

Conclusión

Con este examen se comprobó que en una conexión TCP el orden de los mensajes debe estar coordinado entre ambos programas: para que el cliente escriba primero fue necesario cambiar el orden de send() y recv() en los dos extremos. También se observó la utilidad de recibir el puerto por línea de comandos y de revisar los errores de cada llamada, ya que permitió resolver el conflicto con el puerto 5000 sin modificar el código. Tanto la prueba local como la conexión al servidor del profesor se realizaron correctamente.

Referencias

Kerrisk, M. (2010). *The Linux programming interface: A Linux and UNIX system programming handbook*. No Starch Press.

Stevens, W. R., Fenner, B., y Rudoff, A. M. (2004). *UNIX network programming: The sockets networking API* (3.ª ed., Vol. 1). Addison-Wesley.